
Software Testing & Quality Assurance

Exam Short Notes | All 5 Units

Course Code: CS6T002 | Semester VI | B.Tech CSE
JD College of Engineering & Management, Nagpur | 2025-26

Unit 1
Basic Concepts

Unit 2
Unit Testing

Unit 3
Control Flow

Unit 4 & 5
Data Flow & Categories

Unit 1: Basic Concepts of Testing

1.1 Need for Software Testing

Software testing verifies that an application works correctly, performs well, and is free from bugs before reaching users. Its main purpose is to identify errors, faults, or missing requirements in contrast to actual requirements.

1.2 Errors, Faults, Defects & Failures

Term	Definition
Error	A mistake made by a developer due to wrong logic, syntax, or misunderstanding of requirements. Errors are generated due to wrong logic, syntax, or loop.
Fault	An unintended behavior caused by invalid documented steps or lack of data definitions. If untreated, a fault leads to failure.
Defect / Bug	Situation where actual output does not match expected output. Raised when developer makes mistakes during development phase. Most common type: crash.
Failure	Accumulation of several defects that makes the system unresponsive. Detected by end-users post-production when the defective part is executed.

Chain: Error (human mistake) → Fault (code flaw) → Defect (observed mismatch) → Failure (system breakdown)

1.3 Objectives of Testing

Immediate Goals

- Bug Discovery — find errors at any stage of SDLC
- Bug Prevention — prevent duplication of bugs in subsequent phases

Short-Term / Long-Run Goals

- Quality — correctness, integrity, efficiency, reliability
- Customer Satisfaction — thorough testing for end-user happiness
- Reliability — confidence that software will not fail
- Risk Management — reduce failure probability and manage uncertain events

Post-Implementation Goals

- Reduce Maintenance Cost — post-release errors are costlier to fix
- Improved Testing Process — analyze bug history to improve future projects

1.4 Central Issue in Testing

The central issue is the impossibility of exhaustive testing. Software has an astronomically large number of possible inputs and execution paths.

- Infinite test cases: practically impossible to test every combination
- Resource constraints: limited time, budget, personnel
- Balancing risk vs. coverage: prioritize based on criticality and frequency

-
- Evolving software: continuous changes require regression testing

1.5 Testing Activities (STLC)

1. Test Planning — requirement analysis, test strategy, environment setup
2. Test Design & Development — test case and script creation
3. Test Execution — run tests, log defects, retest, regression testing
4. Test Monitoring & Control — track progress, risk management, reporting
5. Test Completion — test summary report, lessons learned

Monitoring activities: Planning → Data Collection → Data Analysis → Reporting → Test Control

1.6 V-Model

V-Model is an SDLC model that integrates testing alongside each development phase, forming a V shape.

Verification Phases (Left Side of V)

1. Business Requirement Analysis — communicate with customer, acceptance test planning
2. System Design — overall system structure (HLD + LLD)
3. Architectural Design — HLD, modules, integration test planning
4. Module Design — LLD, unit test planning
5. Coding Phase — actual code development

Validation Phases (Right Side of V)

1. Unit Testing — eliminate bugs at code/unit level
2. Integration Testing — verify communication between modules
3. System Testing — test complete application (functional + non-functional)
4. User Acceptance Testing (UAT) — verify system meets user requirements in real-world environment

1.7 Test Execution

Test execution verifies that developed code provides expected results per client requirements.

3 Phases of Test Execution

1. Creation of Test Cases — simple, non-repeating, scenario-covering test cases
2. Test Case Execution — automated or manual testing by QA team
3. Validating Test Results — compare actual vs expected, report failures

1.8 Test Tools & Automation

Automation testing uses special software to replace manual testing tasks, integrating with CI/CD pipelines.

Automation Testing Tools

- Selenium — regression testing, web-based, supports Java/Python/Ruby
- QTP (Quick Test Professional) — functional and regression for web/desktop apps
- Sikuli — GUI-based automation using screenshots
- Appium — mobile testing for iOS, Android, Windows
- JMeter — open-source performance and load testing tool

Types of Automated Testing

-
- Unit, Integration, Smoke, Performance, Regression, Security, Acceptance, API, UI

1.9 Limitations of Testing

- Cannot uncover unknown or missing requirements
- Impossible to test all conditions — infinite input combinations
- Gives no insight into root causes of errors
- Limited by time and resource constraints
- Cannot guarantee absolute correctness

Unit 2: Unit Testing

2.1 Concepts of Unit Testing

Unit testing is the process of testing the smallest parts of code (units) in isolation. Tests are written for individual functions/methods to verify they work based on intended logic.

Key Techniques

- Logic checks — valid inputs, all code paths tested
- Boundary checks — normal (e.g., 5), edge case (e.g., 3), invalid (e.g., 9) inputs
- Error handling — verify system handles errors without crashing
- Object-oriented checks — verify object state is correctly updated

Types

- Manual Unit Testing — developer tests by hand; costly, time-consuming
- Automated Unit Testing — scripts run automatically; accurate, fast, widely preferred

2.2 Static Unit Testing

Testing performed WITHOUT executing code. Also called Verification Testing or Non-execution Testing. Done in early stages to avoid errors.

Why Static Testing?

- Early bug detection — cheaper to fix than post-release
- Dynamic testing is expensive and time-consuming
- Improves development productivity

Technique 1: Review

- Informal — creator presents content, audience gives opinion
- Walkthrough — done by an experienced expert to check defects
- Peer Review — colleagues check each other's documents
- Inspection — verification by higher authority (e.g., SRS review)

Technique 2: Static Analysis (Tools)

Evaluates code quality written by developers. Detects:

- Unused variables
- Dead code
- Infinite loops
- Variables with undefined values
- Wrong syntax

Types: Data Flow analysis, Control Flow analysis, Cyclomatic Complexity

2.3 Defect Prevention

Defect prevention ensures that defects detected so far do not appear again. A Coordinator facilitates communication, leads DP efforts, and manages the DP Board.

Key Methods

- Requirement analysis — carefully understand and review software requirements
- Review & Inspection — use self-review and peer-review at all stages
- Root Cause Analysis — identify what triggered the defect and prevent recurrence
- Static Code Analysis — automated tools scan code without execution

-
- Pair Programming — two developers share a workstation for real-time review
 - Test-Driven Development (TDD) — write tests before writing code
 - Training — invest in skill development to reduce common mistakes
 - Checklists — ensure critical steps are not missed during development

2.4 Dynamic Unit Testing

Testing performed BY EXECUTING the code and analyzing its behavior with different input values.

Levels of Dynamic Testing

1. Unit Testing — test individual components/units
2. Integration Testing — test how components work together
3. System Testing — test the entire system against requirements
4. Acceptance Testing — test from end-user perspective before release
5. Performance Testing — evaluate system under specific workloads
6. Security Testing — identify and evaluate security risks

Dynamic Testing Process

1. Test Case Design — define objectives, test data, expected outcomes
2. Test Environment Setup — configure hardware, software, network
3. Test Case Execution — run tests, log actual vs. expected results
4. Test Analysis — evaluate outcomes, document defects

2.5 Mutation Testing

Introduced by Richard Lipton in 1971. Makes small modifications (mutations) to source code to check if existing tests can detect them.

Objectives

- Identify pieces of code not properly tested
- Identify hidden defects not found by other methods
- Calculate the mutation score
- Assess how well test cases perform

3 Types of Mutations

Type	Description & Example
Value Mutations	Constants are changed — e.g., <code>int mod = 1000000007</code> changed to <code>int mod = 1007</code>
Decision Mutations	Logical/arithmetic operators changed — e.g., <code>if(a < b)</code> changed to <code>if(a > b)</code>
Statement Mutations	Statements deleted or replaced — e.g., variable <code>c</code> replaced with variable <code>d</code>

Tools for Mutation Testing

Judy, Jester, Jumble, PIT, MuClipse

2.6 Debugging

Debugging is the process of finding and fixing bugs in software after testing reveals a problem.

6-Step Debugging Process

1. Reproduce the Bug — recreate conditions that caused the bug
2. Locate the Bug — find where the bug is using error messages/logs/tools
3. Identify Root Cause — examine logic and flow of code
4. Fix the Bug — make corrections; may need multiple attempts
5. Test the Fix — Unit Tests, Integration Tests, System Tests, Regression Tests
6. Document the Process — record cause, fix, and important details

Debugging Strategies

- Brute Force — study system at length; analyze recent changes
- Backtracking — trace program backward from failure to find faulty region
- Forward Analysis — trace forward using breakpoints/print statements
- Cause Elimination — binary partitioning of error data to isolate causes
- Static Analysis — analyze code without executing it
- Dynamic Analysis — analyze code behavior at runtime
- Collaborative Debugging — multiple developers debug together
- Logging & Tracing — collect and analyze logs generated during execution
- Automated Debugging — use AI/ML tools to identify and suggest fixes

Testing vs Debugging

Aspect	Testing	Debugging
Definition	Testing finds bugs/errors	
Purpose	Identify defects in software	
Timing	Done before debugging	
Approach	Execute software with test cases	

2.7 Unit Testing Tools

Criteria for Choosing Unit Testing Tools

1. Learning Curve — time investment must be worthwhile
2. Stability & Customer Support — good documentation and online resources
3. Simple GUI — clear reports for quick conclusion of test results
4. Load Testing Capability — support for Excel, XML data input
5. Script Maintenance — record and playback feature
6. Testing Types Supported — Unit, Regression, and more
7. Budget — carefully consider cost

Popular Unit Testing Tools

LambdaTest, JUnit, NUnit, TestNG, PHPUnit, Mockito, Cantata, Karma, Mocha, TypeMock

Unit 3: Control Flow Testing

3.1 Control Flow Testing

A structural/white box testing strategy that uses a program's control flow as a model. Developers test their own code since they know the design and implementation best.

Control Flow Testing Process

1. Control Flow Graph Creation — from source code, manually or using software
2. Coverage Target — define nodes, edges, paths, branches to be covered
3. Test Case Creation — create test cases to cover the defined target
4. Test Case Execution — run the test cases
5. Analysis — determine if program is error-free or has defects

Objectives

- Path Coverage — every path through the program executed at least once
- Branch Coverage — every decision point evaluated as true and false
- Decision Coverage — every decision point executed at least once
- Loop Testing — test zero, single, and multiple iterations
- Error-Handling Path Testing — test error/exception handling paths
- Boundary Value Examination — test behavior at input range limits
- Cyclomatic Complexity Reduction — determine and reduce complexity

3.2 Control Flow Graphs (CFG)

A directed graph representing all execution paths during a program. Developed by Frances E. Allen. Used in static analysis and compiler applications.

Key Characteristics

- Process-oriented — shows all paths traversable during execution
- Directed graph — edges show flow direction
- Nodes represent basic blocks; edges represent control flow paths
- Entry Block — control enters; Exit Block — control leaves

CFG Shapes for Common Constructs

Construct	CFG Description
if-else	Diamond/branch shape — two edges from decision node, merge at end
while loop	Loop back to condition — check condition before body executes
do-while loop	Body executes first, then condition is checked (loop back)
for loop	Init → Condition check → Body → Update → back to Condition

3.3 Paths in Control Flow Graph

A path is a sequence of nodes and edges representing one possible execution route from start to end.

Path Type	Description
Independent Path	Introduces at least one new edge not covered in other paths. Used in Basis Path Testing.
Feasible Path	A path that can actually be executed with some valid input.

Infeasible Path	A path that is logically impossible to execute — no input can trigger it.
------------------------	---

3.4 Path Selection Criteria

Used to choose a set of important paths that give maximum fault detection with minimum test cases.

1. Statement Coverage	Execute every statement at least once. Select paths so all nodes in CFG are visited.
2. Branch Coverage	Execute each branch (True/False) at least once. Select paths making each decision outcome occur.
3. Condition Coverage	Each individual condition in a decision must be both True and False at least once.
4. Path Coverage	Execute all possible paths in the program. Most thorough but often impractical for complex programs.
5. Basis Path Testing	Use Cyclomatic Complexity to find minimum number of independent paths. Most commonly used in practice.

3.5 Cyclomatic Complexity

Measures the number of linearly independent paths in a program. Used to determine minimum test cases needed.

Formula

$$M = E - N + 2P$$

Where: E = number of edges, N = number of nodes, P = number of connected components (usually 1)

Simplified: M = number of decision points + 1

- Higher complexity = more test cases needed
- Used in Basis Path Testing to decide number of independent paths

3.6 Test Data Generation

The process of collecting and managing data to implement test cases. Datasets act as inputs to check actual vs. expected behavior.

Technique	Description, Pros & Cons
Manual	Tester creates all data manually based on experience. Easy, no extra tools. Slow and accuracy is doubtful.
Automated	Tools generate data at high speed by analyzing large volumes. Accurate and fast. Costly to implement.
Back-end Data Injection	SQL queries injected into database to populate required test data. Fast, less expertise needed. Risk of invalid queries corrupting DB.
Third-party Tools	External tools understand system and generate data accordingly. Accurate and customizable. High cost, less generic.

Unit 4: Data Flow Testing

4.1 Introduction to Data Flow Testing

Data Flow Testing is a structural testing method that finds test paths based on WHERE variables are defined and WHERE they are used. It is unrelated to data flow diagrams.

Key Notation

- $DEF(S) = \{X \mid \text{statement } S \text{ contains the definition of } X\}$
- $USE(S) = \{X \mid \text{statement } S \text{ contains the use of } X\}$
- If statement is a loop or if-condition: DEF set is empty, USE set is from condition

Data Anomalies Detected

- Variable defined but never used or referenced
- Variable used but never defined (uninitialized)
- Variable defined twice before it is used

4.2 Data Flow Graphs (DFG)

A graphical representation showing how data moves through a program, where variables are defined and used, and the paths connecting definitions to uses.

Components of a DFG

Component	Meaning
Node	Represents a statement or block of code
Edge	Represents flow of control between statements
Definition (def)	Point where a variable receives a value (e.g., $x = 10$)
Use	Point where a variable's value is accessed
c-use (computational use)	Variable used in a calculation (e.g., $z = x + y$)
p-use (predicate use)	Variable used in a condition/decision (e.g., $\text{if}(x > y)$)
d-u path	A path in the CFG from a variable's definition to its use

Example

For the code: `a=10; b=5; if(a>b) c=a-b; else c=b-a; print(c);`

- Line 2: a — def. Line 3: b — def. Line 4: a, b — p-use
- Line 5: c — def; a, b — c-use. Line 7: c — def; a, b — c-use. Line 8: c — c-use
- d-u paths for variable c: Path 1: Line 5 → Line 8, Path 2: Line 7 → Line 8

4.3 Data Flow Testing Criteria

Rules used to design test cases based on definition and use of variables. All criteria ensure every definition and use is properly tested.

1. All-Definitions (All-defs)	For every variable definition, at least one use (c-use or p-use) must be covered in a test path. Weakest criterion.
2. All c-uses	For every variable definition, all paths leading to a computational use of that variable must be tested.

3. All p-uses	For every variable definition, all paths leading to a predicate (condition) use must be tested.
4. All Uses	Combination of All c-uses + All p-uses. Every definition to every use (both types) must be covered.
5. All du-paths	Test every distinct path from each definition to each use, including all intermediate paths. Strongest criterion.

4.4 Comparison of Data Flow Test Selection Criteria

Subsumption hierarchy (stronger criteria subsume weaker ones):

All du-paths $\supseteq$ All Uses $\supseteq$ All c-uses / All p-uses $\supseteq$ All-defs

Criterion	Strength & Notes
All-defs	Weakest. Only requires one use per definition. Minimum coverage.
All c-uses / All p-uses	Intermediate. Covers only one type of use.
All Uses	Covers both c-uses and p-uses. Most commonly used in practice — good balance of coverage and cost.
All du-paths	Strongest. Tests every possible path from every definition to every use. Highest cost.

Unit 5: Software Testing Categories

5.1 Functional Testing

Verifies that each function of the application works in conformance with requirements. Does NOT concern itself with source code. Primarily Black Box Testing.

4-Step Functional Testing Process

1. Identify Test Input — core features (login, payment), usability functions, error conditions
2. Compute Expected Outcomes — define what should happen for each input based on specifications
3. Execute Test Cases — run manually or using Selenium, JUnit, TestNG
4. Compare Actual vs Expected Output — confirm functionality works correctly

Types of Functional Testing

- System Testing — tests the fully integrated system against requirements
- Regression Testing — verifies code changes haven't broken existing features

5.2 System Testing

Evaluates the complete, fully integrated system against system requirement specifications. Tests design, behavior, and customer expectations.

System Testing Process

1. Test Environment Setup — create proper testing environment
2. Create Test Cases — generate test cases for the system
3. Create Test Data — prepare data required for testing
4. Execute Test Cases — run all test cases
5. Defect Reporting — detect and record defects
6. Regression Testing — check for side effects
7. Log Defects — fix identified defects
8. Retest — re-run if test was unsuccessful

5.3 Regression Testing

Re-executes previously created test suite to verify that recent code changes, bug fixes, or enhancements have NOT broken existing functionality.

Regression Testing Process

1. Identify Code Changes — analyze areas modified (new features, bug fixes, optimizations)
2. Debug and Fix Failures — resolve defects caused by changes
3. Modify Code — apply necessary updates
4. Select Test Cases — choose relevant test cases; add new ones for new functionality
5. Execute Regression Tests — run manually or using automated tools
6. Analyze Results — review outcomes, identify regressions, document issues
7. Retest as Needed — fix defects and re-run to confirm resolution

5.4 Performance Testing

Ensures software applications perform properly under expected workload. Evaluates sensitivity, reactivity, and stability under load. Identifies bottlenecks and verifies scalability.

Type	Description & Objective
Load Testing	Simulates real-world concurrent user load. Finds maximum users/transactions the system can handle. Objective: identify performance congestion before launch.
Stress Testing	Tests system BEYOND normal usage levels. Finds the breaking point. Also called Endurance/Torture Testing. Objective: identify breaking point of the software.
Scalability Testing	Tests system's ability to scale up/down with growing user demand. Helps plan capacity additions. Carries out at hardware, software, or database level.

5.5 Load Testing

Determines system performance under real-life load conditions — how system behaves when multiple users access it simultaneously.

Objectives of Load Testing

- Evaluate scalability — find point where system starts performing badly
- Capacity planning — understand future infrastructure needs
- Determine bottlenecks — find where performance suffers under load
- Response time analysis — ensure reasonable response at all loads
- Find memory leaks — identify resource usage issues over time

Load Testing Metrics

Metric	Description
Average Response Time	Average time to respond to all user requests. Shows application speed.
Error Rate	Percentage of failed requests to total requests. High error rate = low efficiency.
Throughput	Bandwidth consumed; amount of data flowing between user and app server. Measured in KB/s.
Requests Per Second	Number of requests (images, pages, docs) generated to the server per second.
Concurrent Users	Count of users actively present at any time, even without making requests.
Peak Response Time	Longest time taken to handle a request; helps find slowest resources.

Load Testing Process

1. Test Environment Setup — create dedicated load testing environment
2. Load Test Scenario — create scenarios, determine transactions, prepare data
3. Test Scenario Execution — execute scenarios, gather measurements and metrics
4. Test Result Analysis — analyze results, make recommendations
5. Retest — if failed, perform test again

5.6 Stress Testing

Tests the system's robustness and error handling above normal operating levels. Ensures system does not crash under crunch situations.

Purpose of Stress Testing

- Analyze behavior after failure — show appropriate error messages under extreme conditions
- System recovery after failure — ensure recovery plans exist
- Uncover hardware issues and data corruption issues
- Uncover security weaknesses — vulnerabilities during constant peak load
- Ensure data integrity — data must remain reliable even after failure

Stress Testing Process

1. Planning — gather system data, analyze system, define stress test goals
2. Create Automation Scripts — use JMeter or LoadRunner; generate realistic test data
3. Script Execution — run scripts, analyze data, identify bottlenecks
4. Result Analysis — analyze test results, identify bottlenecks, store metrics
5. Tweaking & Optimization — fine-tune system, optimize code, retest to confirm benchmarks

5.7 Scalability Testing

Tests ability to scale up or down to support increasing user load, transactions, or data volumes.

Objectives

- Determine how application scales with increasing workload
- Determine maximum user limit for the software product
- Determine client-side degradation and end-user experience under load
- Assess system performance under various network conditions
- Guarantee acceptable response times and user experience at scale

Scalability Testing Attributes

Attribute	Description
Response Time	Time between user request and application response. Decreases as load increases.
Throughput	Number of requests processed per unit time. Measured differently per app type.
Threshold Load	Number of requests the app can process with desired throughput.
CPU Usage	CPU utilization while executing instructions. Measured in Megahertz.
Memory Usage	Memory consumed for a task. Measured in bytes.
Network Usage	Bandwidth consumed by the application under test. Measured in bytes/sec.

Steps for Scalability Testing

1. Define a repeatable process for executing the test
2. Determine criteria for scalability test
3. Determine software tools required
4. Set up testing environment and configure hardware
5. Create and verify visual scripts
6. Create and verify load test scenarios
7. Execute the test
8. Evaluate results
9. Generate required report

5.8 Testing Documents

Phase	Documents Required
Before Testing	SRS document, Test Policy document, Test Strategy document, Traceability Matrix document
During Testing	Test Case document (Unit/Integration/System/Acceptance test plans), Test Description, Test Case Report, Test Logs
After Testing	Test Summary Report — collective analysis; determines if software is ready to launch

Quick Revision: Key Formulas & Mnemonics

Important Formulas

Cyclomatic Complexity	$M = E - N + 2P$ (E=edges, N=nodes, P=connected components)
Simplified CC	$M = \text{Number of Decision Points} + 1$
Mutation Score	$\text{Killed Mutants} / \text{Total Mutants} \times 100\%$

Key Terms at a Glance

CFG	Control Flow Graph — directed graph of all execution paths
DFG	Data Flow Graph — shows where variables are defined and used
c-use	Computational Use — variable used in a calculation
p-use	Predicate Use — variable used in a condition/branch
STLC	Software Testing Life Cycle — 5 phases from Planning to Completion
V-Model	Verification + Validation model — each dev phase maps to a test phase
TDD	Test-Driven Development — write tests BEFORE writing code
CI/CD	Continuous Integration/Delivery — automated testing in deployment pipeline
UAT	User Acceptance Testing — final validation by end users
SRS	Software Requirements Specification — primary reference for testing

Unit-wise Course Outcomes

Unit / CO	Outcome Statement
Unit 1 — CO1	Understand basics of software testing and its importance
Unit 2 — CO2	Thorough understanding of Unit Testing and Debugging Techniques
Unit 3 — CO3	Identify and evaluate different paths in a CFG using path selection criteria
Unit 4 — CO4	Construct and interpret Data Flow Graphs to analyze data flow
Unit 5 — CO5	Explain functional and performance testing techniques